

EIN092B: Laboratorio PowerBI

1. Objetivos

Al finalizar este laboratorio, el/la estudiante será capaz de:

- Construir un modelo semántico en Power BI con múltiples fuentes.
- Implementar transformaciones robustas en Power Query para limpieza, tipificación y cálculos base.
- Desarrollar medidas DAX (Data Analysis Expressions).
- Crear parámetros *What-if*.
- Diseñar un dashboard profesional multi-página con *slicers*, KPIs y análisis diferenciados.
- Configurar seguridad a nivel de fila (RLS - Row Level Security).

2. Introducción de entrega

Las actividades planteadas en esta guía deben ser entregadas durante la sesión de laboratorio. Para el proceso de revisión consideren lo siguiente:

- La revisión y evaluación de cada una de las actividades se realizará en dos etapas: (i) una validación básica de requerimientos durante la sesión, y (ii) una verificación de requerimientos posterior a la sesión.
- Durante la sesión, avise al staff (profesor o ayudante) cuando complete una o varias actividades para que muestre y explique sus resultados, y responda las preguntas que se le realizarán sobre los conceptos asociados.
- Durante la sesión, el staff marcará en una planilla cuando Ud. considere una actividad terminada y haya pasado la interrogación. El tener la actividad marcada como revisada en la sesión no entrega puntaje, y solo asegura que cumple con requerimientos para que entregue sus archivos y pase a la siguiente etapa de revisión.
- En el caso de las actividades no entregadas en la sección correspondiente contará como no entregada a tiempo y **deberá entregar la actividad corregida en la siguiente sesión, aplicando descuentos por atraso (-15 %)**.

2.1. Indicaciones para la entrega

La entrega se realizará a través de AulaUSM, Se permitirá el envío de los archivos hasta las **23:59 del 26 de agosto**. Sin embargo, solo se revisarán las actividades marcadas como revisadas durante la sesión. La entrega deberá incluir:

1. El archivo **.pbix** final.
2. Documento PDF breve que explique:
 - Decisiones de modelado.
 - Principales transformaciones en Power Query.
 - Medidas DAX desarrolladas.
 - Implementación de RLS.

3. Fundamentos teóricos

Antes de comenzar con las actividades prácticas, es importante manejar los conceptos que sustentan el trabajo en Power BI. Estos conceptos no solo explican qué se hace en cada etapa del laboratorio, sino también por qué se hace de esa manera, y sientan además la base para herramientas y prácticas de *Business Intelligence* (BI).

3.1. Business Intelligence y Power BI

Business Intelligence (BI) es el conjunto de procesos, herramientas y tecnologías orientadas a transformar datos crudos en información útil para la toma de decisiones. Power BI es una herramienta de *self-service BI*: permite recorrer todo el flujo sin depender exclusivamente de equipos especializados en TI. Este flujo típicamente sigue las etapas:

- **Obtención de datos:** Conexión a distintas fuentes (Excel, bases de datos, archivos planos, servicios web).
- **Transformación (ETL):** Limpieza y preparación de los datos mediante Power Query (equivalente al proceso de *Extract, Transform, Load*).
- **Modelado semántico:** Definición de tablas, relaciones y medidas que dan significado analítico a los datos.
- **Visualización y consumo:** Construcción de reportes y dashboards interactivos para el usuario final.

Un **modelo semántico** (antes llamado *dataset* en Power BI) es la capa que conecta los datos transformados con las visualizaciones: define qué tablas existen, cómo se relacionan entre sí y qué cálculos (medidas) están disponibles para explorar. Es la pieza angular de cualquier proyecto en Power BI, ya que un modelo mal diseñado puede generar resultados incorrectos incluso si las visualizaciones se ven correctas.

3.2. Modelado dimensional

La forma más común y recomendada de estructurar un modelo semántico es el **esquema estrella** (*star schema*), que distingue dos tipos de tablas:

- **Tablas de hechos (*fact tables*):** Contienen las transacciones o eventos medibles (ej. ventas, montos, cantidades) y suelen tener muchas filas.
- **Tablas de dimensiones (*dimension tables*):** Contienen atributos descriptivos utilizados para filtrar y agrupar los hechos (ej. productos, clientes, fechas, regiones).

En un esquema estrella, las tablas de dimensiones se conectan directamente a la tabla de hechos, evitando relaciones indirectas entre dimensiones. Un concepto cercano (aunque menos recomendado en Power BI por afectar al rendimiento) es el **esquema copo de nieve** (*snowflake schema*), donde las dimensiones se normalizan en subtablas adicionales. Al definir relaciones entre tablas es clave entender dos propiedades:

- **Cardinalidad:** Describe cuántas coincidencias existen entre los valores de dos tablas relacionadas (uno a muchos, uno a uno, muchos a muchos). La relación uno a muchos, desde una dimensión hacia la tabla de hechos, es la más común y recomendada.
- **Dirección del filtro:** Determina si los filtros aplicados en una tabla se propagan solo hacia una tabla relacionada (*single direction*) o hacia ambas (*both/bidirectional*). El filtrado bidireccional puede ser útil, pero también puede introducir ambigüedades o afectar el rendimiento si se usa sin criterio.

3.3. Transformación de datos

Power Query es el motor de transformación de datos de Power BI, y cada acción realizada en su interfaz gráfica (eliminar columnas, cambiar tipos, filtrar filas) genera automáticamente código en el lenguaje **M**, un lenguaje funcional diseñado para describir secuencias de transformaciones sobre datos. Cada transformación queda registrada como un **paso** (*applied step*), lo que permite revisar, reordenar o modificar el proceso de limpieza en cualquier momento, de forma similar a un historial de cambios reproducible.

Algunas transformaciones habituales en esta etapa incluyen: tipado de columnas (texto, fecha, número), eliminación de duplicados y filas vacías, normalización de nombres de columnas, creación de columnas calculadas simples y filtrado de registros irrelevantes. Realizar estas transformaciones en Power Query, en lugar de directamente en el modelo, es una buena práctica porque mantiene el modelo semántico más liviano y evita cálculos innecesarios en tiempo de consulta.

3.4. Lenguaje DAX para construcción de medidas

DAX (*Data Analysis Expressions*) es el lenguaje de fórmulas utilizado en Power BI para crear cálculos dentro del modelo semántico. Es importante distinguir dos tipos de cálculos:

- **Columnas calculadas:** Se calculan fila por fila al procesar el modelo y quedan almacenadas físicamente en la tabla, ocupando espacio en memoria.
- **Medidas:** Se calculan dinámicamente en el momento de la consulta, según el contexto de filtros activo (por ejemplo, el año o la región seleccionados en un slicer). Son la forma preferida de construir KPIs, ya que no aumentan el tamaño del modelo y se recalculan automáticamente según la interacción del usuario.

El comportamiento de las medidas depende de dos conceptos centrales en DAX:

- **Contexto de fila:** El contexto que existe al evaluar una expresión fila por fila (relevante, por ejemplo, en columnas calculadas).
- **Contexto de filtro:** El conjunto de filtros activos al momento de evaluar una medida, ya sea por slicers, filtros de página o filas/columnas de una tabla o matriz.

La función **CALCULATE()** es una de las más importantes de DAX porque permite modificar el contexto de filtro de una expresión, y es la base de funciones más avanzadas como **ALL()**, **ALLSELECTED()** y **FILTER()**, utilizadas para construir medidas más complejas (comparaciones interanuales, rankings, porcentajes del total, entre otras).

3.5. Parámetros What-if para el análisis de escenarios

Los **parámetros What-if** permiten simular distintos escenarios dentro de un reporte, dando al usuario final la posibilidad de ajustar variables (como un porcentaje de descuento o una tasa de crecimiento) y observar en tiempo real cómo cambian los resultados. Técnicamente, un parámetro What-if genera una tabla auxiliar **desconectada** del modelo (*disconnected table*), es decir, sin relaciones con las tablas de hechos o dimensiones, cuyo único propósito es alimentar una medida a través de funciones como **SELECTEDVALUE()**, y no filtrar ni agregar datos como lo haría una dimensión normal.

Este mecanismo es un caso particular de algo más general en BI: el **análisis de sensibilidad** (*sensitivity analysis*), que evalúa cómo varía un resultado ante cambios en una o más variables de entrada. Dentro de este concepto, se pueden encontrar ciertas variantes:

- **Análisis de una variable:** Es lo que construye un parámetro What-if estándar en Power BI, ajustando una sola variable (ej. descuento) y observando el efecto sobre una o varias medidas.
- **Análisis de escenarios discretos (*scenario analysis*):** En lugar de un rango continuo, se comparan un número reducido de escenarios predefinidos (ej. *pesimista*, *base*, *optimista*), cada uno con su propia combinación de supuestos.

- **Análisis multivariable:** Evalúa el efecto combinado de dos o más variables simultáneamente (ej. descuento y volumen de ventas a la vez). En Power BI esto suele requerir combinar varios parámetros What-if o recurrir a visuales tipo *tornado chart*.

Es importante aclarar que un parámetro What-if no reemplaza una medida existente, sino que crea una nueva variable independiente que luego se combina con medidas ya definidas. Por eso es clave definir primero qué medidas base existen antes de construir un escenario sobre ellas.

3.6. Indicadores clave de desempeño (KPIs)

Un **KPI** (*Key Performance Indicator*) es una medida cuantificable que refleja qué tan bien se está cumpliendo un objetivo de negocio específico. No todo número relevante es un KPI: para que una medida sea considerada un buen indicador de desempeño, suele evaluarse contra criterios como los del marco **SMART** (*Specific, Measurable, Achievable, Relevant, Time-bound*), es decir, que sea específica, medible, alcanzable, relevante para el negocio y acotada en el tiempo.

Toda métrica es un dato medible (ej. cantidad de visitas a una página), pero solo se convierte en KPI cuando está directamente ligada a un objetivo estratégico y tiene una meta de referencia definida. Por eso, en la práctica, un KPI casi siempre requiere tres componentes:

- **Valor actual:** El resultado obtenido hasta el momento (ej. Ventas Totales).
- **Objetivo (*target*):** El valor esperado o comprometido contra el cual se compara.
- **Estado o tendencia (*status/trend*):** Una señal visual (color, ícono, flecha) que indica si el indicador va bien, mal, o cómo ha evolucionado en el tiempo.

Esta estructura de tres componentes es, de hecho, la que sigue el visual nativo **KPI** de Power BI, y también puede construirse de forma más flexible combinando tarjetas, medidas de estado y formato condicional. Al seleccionar qué KPIs incluir en un dashboard, conviene considerar además:

- **Indicadores adelantados y rezagados (*leading vs. lagging*):** Los indicadores rezagados miden resultados ya ocurridos (ej. ventas del mes pasado), mientras que los adelantados intentan anticipar resultados futuros (ej. cantidad de cotizaciones abiertas). La combinación de ambos permite observar tanto el desempeño alcanzado como las señales que pueden influir en su evolución.
- **Evitar las *vanity metrics*:** Son indicadores que se ven bien o añaden atractivo visual pero no informan realmente decisiones (ej. número total de registros históricos sin relación con ningún objetivo ni contexto específico). Un KPI debe responder a una necesidad concreta y facilitar la identificación de posibles acciones.

3.7. Principios de diseño de dashboards

Un dashboard efectivo no es solo una colección de gráficos: debe comunicar información de forma clara y jerarquizada. Algunos principios generales, cercanos a los vistos en visualización de datos, aplican directamente al diseño en Power BI:

- **Jerarquía visual:** Los KPIs e indicadores más relevantes deben ubicarse en las zonas de mayor atención y su número debe mantenerse acotado; el resto de la información puede quedar disponible mediante interacción (drill-down, tooltips, páginas secundarias).
- **Consistencia:** Uso coherente de colores, tipografías y escalas entre páginas, evitando que un mismo color represente cosas distintas en distintos gráficos.
- **Interactividad con propósito:** Slicers y filtros deben facilitar la exploración, no sobrecargar al usuario con demasiadas opciones simultáneas.
- **Elección del gráfico adecuado:** Cada tipo de visualización (líneas, barras, dispersión, mapas) comunica mejor cierto tipo de relación (tendencia, comparación, correlación, distribución geográfica), por lo que su elección no debe ser arbitraria.

3.8. Seguridad y gobierno de datos

A medida que un reporte se comparte con más usuarios, surge la necesidad de controlar qué información puede ver cada uno. La **seguridad a nivel de fila** (*Row Level Security*, RLS) permite definir **roles**, cada uno asociado a una expresión DAX que filtra las filas visibles según el usuario que consulta el reporte. Conviene distinguir dos formas de implementar RLS:

- **RLS estático:** El filtro queda fijo en la definición del rol (ej. `[Region] = "Norte"`). Es simple, pero requiere crear y mantener un rol distinto por cada categoría a restringir, lo cual escala mal si hay muchas regiones o clientes.
- **RLS dinámico:** El filtro se calcula en función de quién está consultando el reporte, típicamente mediante `USERPRINCIPALNAME()` combinado con `LOOKUPVALUE()` sobre una tabla de mapeo usuario-categoría. Un mismo rol sirve para todos los usuarios, y basta con mantener actualizada la tabla de mapeo. Es el enfoque recomendado cuando el número de perfiles de acceso es grande o cambia con frecuencia.

Además, hay dos comportamientos de RLS que conviene tener claros:

- **Los roles son aditivos, no restrictivos entre sí.** Si un mismo usuario pertenece a más de un rol (ej. `Rol_Ventas_Norte` y `Rol_Ventas_Sur`), Power BI le mostrará la unión de las filas permitidas por ambos roles, no la intersección. Asignar a un usuario a más roles de los necesarios amplía su acceso, no lo restringe.
- **RLS depende de la dirección del filtro en el modelo.** El filtro definido en un rol se propaga desde la tabla donde se define hacia las tablas relacionadas siguiendo la dirección de filtrado configurada en las relaciones. Si una relación es de un solo sentido y el filtro de RLS necesita propagarse en el sentido contrario, puede ser necesario habilitar filtrado bidireccional en esa relación específica, evaluando el impacto en rendimiento y en la lógica de otras medidas del modelo.

Un concepto cercano, aunque no se trabaje en este laboratorio, es la **seguridad a nivel de objeto** (*Object Level Security*, OLS), que en lugar de restringir filas, oculta tablas o columnas completas según el rol del usuario (por ejemplo, ocultar una columna de sueldos a ciertos perfiles). Ambos mecanismos forman parte de lo que en BI se conoce como **gobierno de datos**: el conjunto de políticas que aseguran que la información correcta llegue únicamente a quien debe acceder a ella.

Finalmente, vale la pena mencionar que la mayoría del trabajo de este laboratorio se realiza en **Power BI Desktop**, la aplicación de autoría. Sin embargo, en un flujo de trabajo real, los reportes suelen publicarse en el **Power BI Service**, la plataforma en la nube donde además es posible configurar actualizaciones automáticas de datos (*scheduled refresh*) y *gateways* para conectar con fuentes locales. Al publicar, es importante no confundir los **roles de RLS** definidos en el modelo para controlar qué filas de datos ve cada usuario con los **roles del workspace** (Visor, Colaborador, Miembro, Administrador) definidos en el Service para controlar qué puede hacer cada usuario con el reporte en sí (como editarlo o compartirlo). Son dos capas de seguridad distintas y complementarias.

4. Actividades

En este laboratorio, el objetivo es desarrollar un proyecto completo en Power BI que abarque desde la importación de datos hasta la implementación de seguridad a nivel de fila (RLS). Para cumplir con los objetivos planteados, el/la estudiante debe realizar las siguientes actividades:

4.1. Construcción del modelo semántico

En esta etapa, el objetivo es crear la base estructural del proyecto en Power BI, estableciendo un modelo de datos coherente y optimizado para análisis. Para tener más claros los pasos que se les menciona a continuación, considere visitar la siguiente documentación. [Modos de modelo semántico](#)

1. **Seleccionar la base de datos:** Revise las opciones disponibles y escoja una que le resulte interesante o relevante.

- Seleccionar un conjunto de datos tabulares disponible en: <https://mavenanalytics.io/data-playground>, o generar su propio conjunto de datos usando <https://www.mockaroo.com/>. Es importante mencionar que debe seleccionar un conjunto de datos que le permita cubrir con todas las actividades del laboratorio, por esta razón se les recomienda considerar alguno de los dataset de continuación.
 - <https://mavenanalytics.io/data-playground/coffee-shop-sales>
 - <https://mavenanalytics.io/data-playground/pizza-place-sales>
 - <https://mavenanalytics.io/data-playground/video-game-sales>
- 2. **Importar datos desde múltiples fuentes:** Utilice la pestaña **Inicio** → **Obtener datos** para conectar al menos dos orígenes distintos. Ejemplos:
 - Archivo Excel con datos de ventas históricas.
 - Base de datos SQL Server con información de clientes.
 - CSV con catálogos de productos.
 - Fuente Web para tipos de cambio o indicadores económicos.
- 3. **Comprobar estructura de las tablas importadas:** Antes de proceder, verifique en el panel de *Vista de datos* que:
 - Todas las columnas se hayan cargado correctamente.
 - No existan columnas innecesarias (pueden eliminarse en Power Query).
 - Los nombres de las columnas sean claros y sin caracteres especiales.
- 4. **Definir relaciones entre tablas:** Acceda a la *Vista de modelo* (parece un diagrama con cajitas) y establezca relaciones explícitas entre las tablas, arrastrando los campos clave. Ejemplo de relación esperada:

Tabla Ventas (muchos) --- (uno) Tabla Productos
Tabla Ventas (muchos) --- (uno) Tabla Clientes
- 5. **Configurar cardinalidades y dirección de filtro:** Asegúrese de que las relaciones estén en la cardinalidad correcta (1 a muchos, 1 a 1, etc.) y que la dirección de filtro sea la más adecuada para el análisis. *Tip:* En la mayoría de los casos, las relaciones deben filtrar en un solo sentido desde las tablas de dimensiones hacia las tablas de hechos.
- 6. **Validar el modelo con una tabla temporal:** Cree una tabla visual simple en una página de prueba para verificar que los datos de diferentes tablas se combinan correctamente. Para tener más claros este paso, considere visitar las siguientes respuesta de Fabric Community. [Modos de modelo semántico Temp Table](#)

4.2. Transformaciones en Power Query

En esta etapa, el objetivo es limpiar, estructurar y preparar los datos para que el modelo semántico funcione correctamente y las medidas DAX posteriores devuelvan resultados coherentes. Para tener más claros los pasos que se les menciona a continuación, considere visitar la siguiente documentación. [Transformación, definición y modelado de datos](#)

1. **Acceder al editor de Power Query:** Desde Power BI Desktop, vaya a **Inicio** → **Transformar datos**. Esto abrirá el editor de Power Query.
2. **Explorar la vista previa de los datos:** Antes de modificar, observe:
 - Si existen columnas innecesarias que puedan eliminarse.
 - Si hay valores nulos, vacíos o inconsistentes.
 - Si los nombres de las columnas son claros y consistentes.

3. **Eliminar duplicados y filas en blanco:** - Seleccione la columna clave de la tabla y use **Quitar duplicados**. - Use el filtro para eliminar filas en blanco o valores nulos.
4. **Corregir tipos de datos:** - Asegúrese de que las columnas numéricas, de fecha y texto tengan el tipo correcto. - Ejemplo en código M:

```
= Table.TransformColumnTypes(Fuente,  
    {"FechaVenta", type date}, {"Cantidad", Int64.Type})
```

5. **Normalizar nombres de columnas:** Cambie nombres largos o con caracteres especiales por versiones claras y sin espacios, usando **Usar primera fila como encabezados** si es necesario.
6. **Crear columnas calculadas básicas:** Ejemplos:

- Concatenar nombre y apellido:

```
NombreCompleto = [Nombre] & " " & [Apellido]
```

- Extraer el año de una fecha:

```
AñoVenta = Date.Year([FechaVenta])
```

7. **Filtrar datos irrelevantes:** Si la tabla contiene información fuera del rango de fechas o categorías de interés, aplique filtros para reducir el volumen y optimizar el rendimiento.
8. **Revisar pasos aplicados:** Verifique la lista de pasos en el panel derecho para confirmar que las transformaciones están en el orden correcto y que no hay pasos redundantes.
9. **Cerrar y aplicar cambios:** Use **Cerrar y aplicar** para guardar las transformaciones y volver a Power BI Desktop.

4.3. Desarrollo de medidas DAX

En esta fase, el objetivo es crear medidas (calculadas con el lenguaje DAX) que permitan obtener métricas clave de forma dinámica y reutilizable en distintas visualizaciones. Para tener más claros los pasos que se les menciona a continuación, considere visitar la siguiente documentación. [Escribe consultas DAX](#)

1. **Identificar las métricas clave:** Revise los datos disponibles y determine qué indicadores son relevantes para el análisis. Ejemplos:
 - Ventas totales.
 - Margen de ganancia (en valor y en porcentaje).
 - Cantidad de clientes únicos.
 - Ticket promedio por cliente.
 - Crecimiento interanual.
2. **Crear las medidas en la tabla de hechos o en una tabla de medidas:** Recomendación: Cree una tabla especial llamada **Medidas** usando **Inicio** → **Nueva tabla** y así agrupar todas las medidas para mantener orden.
3. **Medidas básicas:** Ejemplos:

```
Ventas Totales = SUM(Ventas[Monto])  
Margen Valor = SUM(Ventas[Margen])  
Margen % = DIVIDE(SUM(Ventas[Margen]), SUM(Ventas[Monto]), 0)  
Clientes Únicos = DISTINCTCOUNT(Ventas[ClienteID])
```

4. **Medidas derivadas:** Ejemplo de ticket promedio:

```
Ticket Promedio = DIVIDE([Ventas Totales], [Clientes Únicos], 0)
```

Ejemplo de crecimiento interanual:

```
Crecimiento % =  
DIVIDE(  
    [Ventas Totales] -  
    CALCULATE([Ventas Totales],  
        DATEADD('Calendario'[Fecha], -1, YEAR)),  
    CALCULATE([Ventas Totales],  
        DATEADD('Calendario'[Fecha], -1, YEAR)),  
    0  
)
```

5. **Usar funciones de contexto:** Explore funciones como FILTER, ALL, CALCULATE para crear medidas que respondan a filtros específicos. Ejemplo: Ventas solo del año actual:

```
Ventas Año Actual =  
    CALCULATE([Ventas Totales], YEAR('Calendario'[Fecha]) = YEAR(TODAY()))
```

6. **Validar las medidas:** Cree tablas visuales temporales para comprobar que cada medida devuelve el valor esperado en distintos contextos de filtrado.

7. **Nombrar y formatear correctamente:** - Use nombres claros y consistentes. - Asigne formatos adecuados (moneda, porcentaje, número entero).

4.4. Creación de parámetros What-if

En esta etapa, el objetivo es agregar interactividad al modelo permitiendo la simulación de escenarios, como cambios en precios, descuentos, costos o márgenes. Para tener más claros los pasos que se les menciona a continuación, considere visitar la siguiente documentación. [Creación y uso de parámetros What-if](#)

1. **Definir el escenario a simular:** Determine qué variable de negocio desea ajustar dinámicamente. Ejemplos:

- Porcentaje de descuento en ventas.
- Incremento de precios.
- Variación de costos de producción.

2. **Crear el parámetro What-if:** - En Power BI Desktop, vaya a **Modelado** → **Parámetro** → **Parámetro What-if**. - Defina:

- Valor mínimo.
- Valor máximo.
- Incremento (paso).
- Valor predeterminado.

- Ejemplo para un descuento de 0 % a 30 % en pasos de 1 %:

```
Descuento WhatIf = GENERATESERIES(0, 0.3, 0.01)
```

Esto generará automáticamente:

- Una tabla con los valores del parámetro.
 - Una medida asociada al valor seleccionado por el usuario.
 - Un segmentador (slicer) para controlar el valor en el reporte.
3. **Crear una medida que utilice el parámetro:** Ejemplo aplicando el descuento seleccionado a las ventas:

```
Ventas con Descuento =  
[Ventas Totales] *  
(1 - 'Descuento WhatIf'[Descuento WhatIf Value])
```

4. **Probar la funcionalidad:** - Inserte un slicer con el campo del parámetro. - Cree una tarjeta que muestre [Ventas con Descuento]. - Cambie el valor en el slicer y observe cómo la medida se ajusta.

4.5. Diseño del dashboard

En esta fase, el objetivo es crear un reporte profesional, organizado en varias páginas, que permita explorar la información de forma visual, interactiva y adaptada a distintos tipos de análisis. Para tener más claros los pasos que se les menciona a continuación, considere visitar la siguiente documentación. [*Creación de un panel de Power BI*](#)

1. **Definir la estructura del dashboard:** Cree un diseño de tres páginas:
- **Visión general:** KPI principales, tendencias globales y filtros generales.
 - **Análisis por producto:** ventas, rentabilidad y evolución por categoría o producto.
 - **Análisis por cliente o región:** distribución geográfica, clientes clave y comportamiento de compra.
2. **Diseñar la página de visión general:** - Agregue tarjetas con KPIs como:
- Ventas Totales.
 - Margen %.
 - Clientes Únicos.
 - Crecimiento %.

- Ejemplo de medida para KPI binario:

```
KPI Ventas vs Meta =  
IF([Ventas Totales] >= [Meta Ventas],  
"Meta alcanzada",  
"Meta no alcanzada")
```

- Agregue un gráfico de líneas para la tendencia mensual. - Inserte slicers globales para filtrar por año, región o categoría.

3. **Diseñar la página de análisis por producto:** - Utilice gráficos de barras o columnas para comparar ventas por categoría o producto. - Añada un gráfico de dispersión para analizar relación entre precio promedio y ventas. - Cree filtros específicos para productos o familias de productos. - Ejemplo de medida para ranking:

```
Ranking Producto =  
RANKX(ALL(Productos[NombreProducto]), [Ventas Totales], , DESC)
```

4. **Diseñar la página de análisis por cliente o región:** - Inserte un mapa para representar ventas por ubicación geográfica. - Agregue tablas con clientes ordenados por compras. - Incluya un gráfico de embudo para representar el proceso de conversión (si aplica). - Ejemplo de medida para ventas acumuladas por cliente:

```
Ventas Acumuladas Cliente =  
    CALCULATE(  
        [Ventas Totales],  
        FILTER(  
            ALLSELECTED(Clientes[ClienteID]),  
            Clientes[ClienteID] <= MAX(Clientes[ClienteID])  
        )  
    )
```

5. **Agregar segmentaciones y visualizaciones condicionales:** - Use segmentaciones para permitir filtrado rápido (ej. botones de categorías). - Aplique formato condicional en tablas y matrices para resaltar valores fuera de lo esperado. - Ejemplo de formato condicional DAX:

```
Color KPI Ventas =  
    IF([Ventas Totales] >= [Meta Ventas], "Green", "Red")
```

6. **Optimizar la experiencia visual:** - Mantenga consistencia en colores, tipografías y tamaños. - Alinee y distribuya elementos para lograr un diseño limpio. - Use títulos descriptivos y nombres claros para cada visual.
7. **Probar la interactividad:** - Aplique diferentes combinaciones de filtros y verifique que todas las páginas respondan correctamente. - Revise que los KPIs cambien de forma coherente.

4.6. Configuración de Row Level Security

El objetivo de esta etapa es implementar seguridad a nivel de fila para restringir el acceso a los datos según el usuario que consulte el reporte. Para tener más claros los pasos que se les menciona a continuación, considere visitar la siguiente documentación. [Definir roles y reglas](#)

1. **Analizar la necesidad de RLS:** - Determine qué dimensión o atributo servirá para restringir los datos (ej. región, departamento, segmento de cliente). - Verifique que la tabla contenga la columna adecuada para filtrar (ej. Region, Departamento).
2. **Definir los roles:** - En Power BI Desktop, vaya a **Modelado** → **Administrar roles**. - Cree un nuevo rol para cada perfil de acceso. Ejemplos:
 - Rol_Ventas_Norte
 - Rol_Ventas_Sur
 - Rol_Marketing
3. **Configurar el filtro DAX para cada rol:** - Seleccione la tabla que contiene la columna de control (ej. Region). - Escriba la expresión DAX que filtrará la información. Ejemplo simple para restringir por región:

```
[Region] = "Norte"
```

Ejemplo dinámico basado en el usuario que inicia sesión:



```
[Region] = LOOKUPVALUE(  
    Usuarios[Region],  
    Usuarios[Email], USERPRINCIPALNAME()  
)
```

Donde la tabla **Usuarios** contiene el mapeo **Email** → **Region**.

4. **Probar la configuración en Power BI Desktop:** - Vaya a **Modelado** → **Ver como**. - Seleccione uno de los roles definidos y verifique que los datos visibles correspondan a lo esperado.
5. **Publicar y asignar roles en el servicio Power BI:** - Publique el reporte en el servicio. - Vaya al conjunto de datos en el servicio y seleccione **Seguridad**. - Asigne usuarios o grupos de Azure AD a cada rol creado.

5. Bonus

A continuación, se presentan actividades opcionales que otorgarán puntos adicionales a la calificación final del laboratorio.

1. **Integración con Python o R en Power Query:** Incorporar un script en Python o R para realizar una transformación o cálculo avanzado que no sea trivial con las herramientas estándar de Power Query. *Ejemplo:* Aplicar un algoritmo de clustering (K-means) y visualizar los grupos en Power BI.
2. **Automatización con Power Automate:** Configurar un flujo que, a partir de una condición detectada en el dashboard (por ejemplo, ventas inferiores al objetivo), envíe un correo automático o actualice una base de datos externa.
3. **Parámetros dinámicos con vinculación a datos:** Crear un parámetro que se alimente dinámicamente de una tabla y que cambie la lógica de las medidas o filtros aplicados en el dashboard.